Token Highlighter

vLLM-Hook integration · June 2026

Paper. Token Highlighter: Inspecting and Mitigating Jailbreak Prompts for LLMs (arXiv:2412.18171)

Related docs. Gradient score derivation (PDF) · Interactive notebook

Token Highlighter ranks prompt tokens by their contribution to a fixed **affirmation target phrase** under teacher forcing, then optionally **mitigates** jailbreak-style completions by scaling selected **driver** embeddings by $\beta < 1$ at a subsequent prefill. This writeup covers the vLLM-Hook integration: capture, offline scoring, driver selection, and embedding-level mitigation during inference.

Overview

Problem and method

Mitigation alters the K/V cache written for the prompt; decoding proceeds from that cache. The deployed pipeline has four stages:

1. **Affirmation loss** (teacher-forced): $L\_aff(X) = -\Sigma_i \log P(y_i \mid X, y_{<i})$
2. **Per-token influence** via closed-form last-block attribution (`forward_attr`)
3. **Driver selection** by `top_percentage` (α) or `mean_std` (threshold k)
4. **Mitigation** by soft-removing drivers at prefill, then decoding

Scoring uses `forward_attr` only: forward hooks during capture and closed-form matrix operations in the analyzer (no second model load when `weight_bundle` is present).

End-to-end flow

```
capture           analyze           mitigate
generate    ->    (offline)    ->   generate
    |                |                  |
    +-- activations +-- highlighter.pt --+ (same prompt, same run_id)
```

Phase	Component	Output
Capture	`HighlighterWorker`	`highlighter_activations.pt`; optional unmitigated decode
Analyze	`HighlighterAnalyzer`	`highlighter.pt` with scores and driver indices
Mitigate	`HighlighterWorker`	Prefill with β-scaled embeddings; mitigated decode

Artifacts: `{hook_dir}/{run_id}/tp_rank_0/highlighter_activations.pt` and `highlighter.pt`.

Architecture

Components

Component	Role
`HookLLM`	Single GPU engine; forwards mode, run id, hook dir, and config via `extra_args`.
`HighlighterWorker`	Capture hooks, trace I/O, embedding soft-removal at mitigate prefill.
`HighlighterAnalyzer`	Closed-form `forward_attr` from disk traces.

Capture and mitigate share one `HookLLM` instance and one worker class. Mitigation is a later `generate(..., highlighter_mode="mitigate")` call that reuses the capture `run_id`.

Implementation: `workers/highlighter_worker.py`, `analyzers/highlighter_analyzer.py`, `utils/TokenHighlighter/`.

Worker execution model

`HighlighterWorker` is mixed into the vLLM GPU worker. At engine setup, `install_hooks()` initializes state, registers a mitigate **embedding hook**, and wraps `execute_model`. **Capture hooks** (last-block Q/K/V post-RoPE, `h_input`, `h_L`, RoPE probe) install on the first capture request via `_ensure_capture_hooks()`.

Hook layer	Registered	Capture	Mitigate
Embedding soft hook	`install_hooks()`	No-op	Scales driver rows by β
Forward-attr + RoPE probe	First capture step	Records activations	Unused

Each scheduling step runs pre-forward setup, `orig_execute_model(...)`, then post-forward bookkeeping (`_highlighter_execute_model_step`). The analyzer runs in the driver process on disk artifacts.

Configuration flows from `model_configs/token_highlighter/*.json` to `HookLLM._highlighter_config` to `extra_args["highlighter"]`. Prefix caching should be disabled or cleared between capture and mitigate on the same prompt.

Capture and scoring

Capture phase

Purpose. Identify driver tokens and persist traces for offline scoring. Soft-removal is not applied during capture; mitigation applies β scaling.

Procedure.

1. On first capture: install forward-attr hooks on the last decoder block.
2. **Teacher prefill** on `prompt + target[:-1]` (`extended` or `suffix` mode).
3. **Persist** `highlighter_activations.pt`: capture tensors, precomputed g_loss = dL/dh^N, and `weight_bundle` from live weights. `W_U` is not stored (~10 MB artifact vs. hundreds of MB with full unembedding).

4. **Restore** the prompt boundary after extended teacher prefill, then **decode**. Prompt-slot KV reflects unmitigated embeddings.

The analyzer loads activations and `weight_bundle`, runs `compute_grad_influences` (default CPU), and writes `highlighter.pt`.

Capture modes

Mode	Forwards	Condition
`extended` (default)	1	Full teacher sequence fits in one prefill chunk
`suffix`	2+	Chunked prefill; merges prompt and teacher-suffix activations

Suffix fallback emits a warning. Increasing `max_num_batched_tokens` is preferred.

forward_attr attribution

The scorer approximates $\|\partial L\_aff/\partial x_i\|$ by back-propagating through the **complete last decoder block**:

1. $g_j = W\_U^T(p_j - e\_\{y_j\})$ at LM-head input.
2. Optional final-norm Jacobian (`rmsnorm`, `layernorm`, `gemma_rms`).
3. **Value and key paths** to a per-prompt-token gradient. The value path holds attention weights α fixed ($\partial/\partial v_i$); the key path uses the full softmax Jacobian $\delta_{ji} = \alpha_{ji}(l_{ji} - \bar{l}_j)$. Captured Q/K/V are post-RoPE; `W_K`/`W_V` are pre-RoPE, so inverse RoPE (R_i^T) is applied per the capture-time probe.
4. **Query path** and **residual identity term** for boundary token (final prompt token, whose query vector influences affirmation loss).
5. Score for token x_i: L2 norm of the resulting vector.

Fidelity. The last-block residual MLP branch is back-propagated analytically: from g_out = dL/dh_L we form g_mid = g_out + J_Norm2^T J_MLP^T g_out (`compute_mid_boundary_gradient`), so the MLP Jacobian is no longer dropped. The closed form covers gated MLPs (SwiGLU/GeGLU, separate `gate_proj`/`up_proj` or fused `gate_up_proj`) and plain MLPs (GELU/ReLU), including input biases. On Qwen2-1.5B, validated with `examples/validate_scorer_agreement.py` (seeded HuggingFace last-block autograd on vLLM's captured block input): g_out / g_mid relative L2 ≈ 0.16% (cosine ≈ 0.99999), block-input gradient relative L2 ≈ 2.6% (cosine ≈ 0.9996). Formal derivation: `utils/TokenHighlighter/grad_influence.py`.

Scorer validation

Offline comparison against a seeded HuggingFace last-block autograd reference (`examples/validate_scorer_agreement.py`). The reference is seeded with vLLM's captured block input so the comparison measures last-block VJP fidelity, not vLLM-vs-HF forward divergence.

Metric	Qwen2-1.5B (3 prompts)
Spearman ρ	1.0
Kendall τ	≈ 1.0
Pearson r	≈ 0.99999
Driver Jaccard (top-α)	1.0

Metric	Qwen2-1.5B (3 prompts)
$\|\Delta g\|_2 / \|g\|_2$ at h_L / h_mid	≈ 0.16%
$\|\Delta g\|_2 / \|g\|_2$ at block input h^(L−1)	≈ 2.6%

Revalidate per model and prompt distribution before deployment.

Mitigation

Purpose. Regenerate the same prompt with driver embeddings scaled by β < 1, writing mitigated K/V into the paged cache.

Prerequisite. `highlighter.pt` from capture and analyze; the same `run_id` as capture.

Procedure.

1. `_load_scores()` reads `highlighter.pt`.
2. `_queue_soft_embeddings()` scales driver embedding rows by β.
3. The embedding hook applies softened rows before layer execution on prefill.
4. Standard prefill and decode run; forward-attr hooks are not re-engaged.

Configuration and API

Per-model defaults: `model_configs/token_highlighter/<model_short>.json`.

Example configuration

```
{
    "model_info": { "name": "Qwen/Qwen2-1.5B-Instruct" },
    "highlighter": {
        "target_phrase": "Sure! I can help with that",
        "target_token_ids": null,
        "capture": "extended",
        "mode": "top_percentage",
        "alpha": 0.25,
        "threshold_k": 2.0,
        "beta": 0.1,
        "require_attn_metadata": false,
        "allow_prerope_fallback": false,
        "reselect_drivers": false
    }
}
```

highlighter block

Field	Default	Description
`target_phrase`	`"Sure! I can help with that"`	Affirmation string for teacher forcing.

Field	Default	Description
`target_token_ids`	unset	Token id list; overrides `target_phrase`.
`capture`	`"extended"`	`"extended"` or `"suffix"`.
`mode`	`"top_percentage"`	Driver selection mode.
`alpha`	`0.25`	Top fraction of prompt tokens flagged as drivers.
`threshold_k`	`2.0`	Mean+std threshold for `mean_std` mode.
`beta`	`0.4`	Mitigate embedding scale ($\beta < 1$ suppresses drivers).
`require_attn_metadata`	`false`	Require attention forward metadata for Q/K/V hooks.
`allow_prerope_fallback`	`false`	Allow pre-RoPE hooks if post-RoPE hooks fail.
`reselect_drivers`	`false`	Recompute drivers at mitigate time.

Per-request and analyzer parameters

Parameter	Role
`highlighter_mode`	`"capture"` or `"mitigate"` (required).
`run_id`	Artifact directory key; must match capture for mitigate.
`scores_run_id`	Alternate run id for `highlighter.pt`.

`HookLLM.analyze(analyzer_spec={...})` merges `mode`, `alpha`, `threshold_k`, and `beta` from the loaded JSON when omitted.

Minimal API sequence

```
llm = HookLLM(model=..., worker_name="token_highlighter",
analyzer_name="token_highlighter", ...)

out_cap = llm.generate(prompt, highlighter_mode="capture",
temperature=0.0, max_tokens=32)
capture_run_id = llm._last_run_id

stats = llm.analyze(analyzer_spec={"top_k": 5})

out_mit = llm.generate(prompt, highlighter_mode="mitigate",
run_id=capture_run_id, temperature=0.0, max_tokens=32)
```

Demos: `examples/demo_token_highlighter.py`, `notebooks/demo_token_highlighter.ipynb`, `notebooks/demo_token_highlighter_colab.ipynb`.

Support and limitations

Intended model coverage

Decoder-only causal transformers with pre-norm blocks, optional final norm, and `lm_head`: LLaMA, Mistral, Qwen, Vicuna, Phi, GPT-2, OPT-shaped, and Gemma-family checkpoints. Demos assume `tensor_parallel_size=1`.

Limitations

Area	Limitation
Architecture	No encoder-decoder, Mamba/RWKV-only, or incompatible custom attention.
forward_attr	Last-block only; MLP Jacobian included analytically (gated/plain MLPs, `nn.Linear`-style weights); GPT-2 `Conv1D` MLPs unsupported.
Tensor parallel	No merged `weight_bundle` across ranks.
Quantization	Hooked modules must expose readable weights.
Mitigation	Task-specific (α, β, affirmation phrase); not a general safety filter.

Open items

- TP-aware `weight_bundle` merge
- Broader norm and block discovery for new vLLM model classes

---